



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

Tema 68. Entorno de desarrollo PHP

AUTOR: Sergio García Tobar

Actualización 2017

ÍNDICE

1	INTRODUCCIÓN	3
2	HISTORIA DE PHP	4
2.1	PHP-FI	4
2.2	PHP 3.....	4
2.3	PHP 4.....	5
2.4	PHP 5.....	5
2.5	PHP 6.....	5
2.6	PHP 7.....	6
3	EL LENGUAJE PHP	7
3.1	CARACTERÍSTICAS	7
3.2	LICENCIA PHP	7
3.3	PÁGINAS PHP	7
3.4	FUNCIONES HTTP.....	8
3.5	CONSTANTES.....	9
3.6	VARIABLES	9
3.7	TIPOS DE DATOS	11
3.8	OPERADORES	11
3.9	SENTENCIAS DE CONTROL.....	11
3.10	ORIENTACIÓN A OBJETOS	12
3.11	FUNCIONES.....	14
3.12	TRATAMIENTO DE EXCEPCIONES.....	15
3.13	CONEXIÓN CON BASES DE DATOS	15
3.14	PHP-CLI	17
4	ENTORNOS PHP.....	18
4.1	TECNOLOGÍAS PARA LA EJECUCIÓN DE PHP	18
4.2	ARQUITECTURA LAMP	19
4.3	FRAMEWORKS DE DESARROLLO	20
4.4	ENTORNOS DE DESARROLLO INTEGRADO	22
4.5	ACELERADORES PHP	22

1 Introducción

La aparición a principio de la década de los noventa del servicio WWW (World-Wide Web) supuso una verdadera revolución en el campo de la informática y las telecomunicaciones. Gracias a WWW, Internet experimentó un gran crecimiento y se crearon nuevas formas de negocio en Internet.

Para poder aprovechar todas las posibilidades que ofrecía Internet fue preciso desarrollar nuevas tecnologías web, tanto en el lado del cliente como en el del servidor. Era preciso que los sitios web ofreciesen contenido dinámico e interactivo. De esta manera, surgieron las páginas web dinámicas y lenguajes como PHP, que supuso, como gran novedad frente a otras tecnologías previas como CGI, el tratarse de un lenguaje embebido en las propias páginas web.

También ligada al desarrollo de Internet apareció la corriente "open source" en la que se desarrolla PHP y otros productos como Linux, Apache y MySQL, con los que PHP pronto se integró perfectamente, de tal manera que se acuñó el término de desarrollo de aplicaciones LAMP (Linux-Apache-MySQL-PHP).

La popularidad alcanzada por PHP le llevó a convertirse en la tecnología favorita para el desarrollo en el lado del servidor, preeminencia que se mantiene en la actualidad. Javascript como tecnología en el lado del cliente y PHP en el lado del servidor son los más utilizados. A fecha de 1 de junio de 2017, según <https://w3techs.com>, el lenguaje de programación más popular del lado del servidor es PHP, con una cuota de mercado del 82,7%, seguido a mucha distancia por ASP.NET con un 14,9% y por Java con un 2,6%. Por su parte, en el buscador Shodan, que analiza la información de las cabeceras HTTP retornadas por los servidores y dispositivos, la búsqueda "php" devuelve cerca de 6 millones de dispositivos (es muy normal encontrar las cabeceras "Server: Apache" y "X-Powered-By: PHP/5").

Está claro que PHP tiene a su favor ser el lenguaje con el que se han construido muchos productos líderes de mercado en sus diferentes sectores como son, por ejemplo, en el sector de los CMS, WordPress, Joomla o Drupal, o plataformas de comercio electrónico como Magento, dentro de una lista muy extensa donde se encuentran también Yahoo!, PHP-Nuke, Wikipedia, Facebook, Owncloud, SugarCRM, Moodle, phpBB o Flickr. Por ello, aunque es cierto que en los últimos años han irrumpido con fuerza lenguajes de servidor como Ruby, Go, Python o Javascript, este último como lenguaje del cada vez más popular entorno NodeJs, no parece que la posición de liderazgo de PHP vaya a verse amenazada en los próximos años.

2 Historia de PHP

2.1 PHP-FI

PHP es el sucesor de un producto anterior llamado PHP/FI, creado en 1994 por Rasmus Lerdorf. La segunda versión de PHP/FI, liberada en 1997, es la base sobre la que se desarrolló PHP3, que se considera la primera versión de este lenguaje.

PHP/FI se desarrolló a partir de un conjunto de scripts CGI escritos por Rasmus Lerdorf en el lenguaje de programación Perl para controlar los accesos a su curriculum online. Llamó a este conjunto de scripts "Personal Home Page Tools", más frecuentemente referenciado como "PHP Tools". A medida que se requería más funcionalidad, Rasmus reescribió PHP Tools, produciendo una implementación mayor y más rica, que proporcionaba un entorno de trabajo para desarrollar aplicaciones web dinámicas sencillas, como libros de visitas, capaces de interactuar con bases de datos. Se refirió a la nueva versión como FI (abreviatura de "Forms Interpreter"). Tenía variables como las de Perl, interpretación automática de variables de formulario y sintaxis incrustada HTML. La sintaxis por sí misma era similar a la de Perl, aunque mucho más limitada, simple y algo inconsistente. De hecho, para embeber el código en un fichero HTML, los desarrolladores tenían que usar comentarios de HTML. Aunque este método no fue bien recibido del todo, FI continuó gozando de expansión y aceptación como una herramienta CGI, pero todavía no completamente como lenguaje.

En octubre de 1995 Rasmus publicó una versión nueva del código. Recordando el nombre PHP, ahora era llamado "Personal Home Page Construction Kit," y fue la primera versión que presumía de ser, en aquel momento, considerada como una interfaz de scripts avanzada. El lenguaje fue deliberadamente diseñado para asemejarse a C en estructura, haciéndolo una adopción sencilla para desarrolladores familiarizados con C, Perl, y lenguajes similares. Habiendo sido así bastante limitado a sistemas UNIX y compatibles con POSIX, el potencial para una implementación de Windows NT estaba siendo explorada.

El código fue completamente rehecho de nuevo, y en abril de 1996, combinando los nombres de versiones anteriores, Rasmus introdujo PHP/FI. Esta implementación de segunda generación comenzó realmente a desarrollar PHP como verdadero lenguaje de programación. Incluía soporte interno para DBM, mSQL, y bases de datos Postgres95, cookies, soporte para funciones definidas por el usuario, y mucho más. Ese mes de junio, PHP/FI brindó una versión 2.0. Sin embargo, un interesante hecho sobre esto, es que sólo había una única versión completa de PHP 2.0. Cuando finalmente pasó de la versión beta en noviembre de 1997, el motor de análisis subyacente ya estaba siendo reescrito por completo. Aunque vivió una corta vida de desarrollo, PHP/FI continuó gozando de un crecimiento de popularidad en el aún joven mundo del desarrollo.

Entre 1997 y 1998, PHP/FI tenía un culto de varios miles de usuarios en todo el mundo. Una encuesta de Netcraft en mayo de 1998 indicó que cerca de 60,000 dominios reportaron que tenían cabeceras que contenían "PHP", indicando en efecto que el servidor host lo tenía instalado. Este número se correspondía con aproximadamente el 1% de todos los dominios de Internet del momento. A pesar de estas impresionantes cifras, la maduración de PHP/FI estaba condenada por las limitaciones en su desarrollo; mientras había varios contribuidores menores, aún era desarrollado principalmente por un individuo.

2.2 PHP 3

PHP 3.0 se liberó oficialmente en junio de 1998. Fue la primera versión que se parecía fielmente al PHP que se conoce hoy en día. Fue creado por Andi Gutmans y Zeev Suraski en 1997 reescribiéndolo completamente a partir de PHP/FI 2.0, al considerarlo ineficiente y carente de las características que necesitaban para su proyecto de desarrollo de una aplicación de comercio electrónico para la universidad. En un esfuerzo para mejorar el motor y comenzar a construir sobre la

base de los usuarios de PHP/FI existentes, Andi, Rasmus y Zeev decidieron colaborar en el desarrollo de un nuevo e independiente lenguaje de programación.

Este lenguaje completamente nuevo fue publicado bajo un nuevo nombre, que eliminó la implicación del uso limitado personal que el nombre PHP/FI tenía. Fue renombrado simplemente como 'PHP', con el significado de un acrónimo recursivo - PHP: Hypertext Preprocessor. De esta manera, PHP3.0 se anunció como el sucesor oficial de PHP/FI 2.0, interrumpiéndose en su mayor parte el desarrollo de PHP/FI 2.0.

Una de las mejores características de PHP 3.0 era su gran extensibilidad. Además de proveer a los usuarios finales de una sólida infraestructura para muchas bases de datos, protocolos y APIs, las características de extensibilidad de PHP 3.0 atrajeron a docenas de desarrolladores a unirse y enviar nuevos módulos de extensión, lo que fue decisivo en el enorme éxito de PHP 3.0. Otras de sus características clave fueron el soporte de sintaxis orientado a objetos y, en general, una sintaxis de lenguaje mucho más potente y consistente.

A finales de 1998, PHP creció hasta una base de instalación de decenas de millares de usuarios estimados y cientos de miles de sitios web. En su apogeo, PHP 3.0 estaba instalado en aproximadamente un 10% de los servidores web de Internet.

2.3 PHP 4

En el invierno de 1998, poco después del lanzamiento oficial de PHP 3.0, Andi Gutmans y Zeev Suraski comenzaron a trabajar en la reescritura del núcleo de PHP. Los objetivos de diseño fueron mejorar la ejecución de aplicaciones complejas, y mejorar la modularidad del código base de PHP. Estas aplicaciones se hicieron posibles por las nuevas características de PHP 3.0 y el apoyo de una gran variedad de bases de datos y APIs de terceros, pero PHP 3.0 no fue diseñado para el mantenimiento tan complejo de aplicaciones eficientemente.

El nuevo motor, apodado "motor Zend" por los apellidos de los autores, alcanzó estos objetivos de diseño satisfactoriamente, y se introdujo por primera vez a mediados de 1999. PHP 4.0, basado en ese motor y provisto de nuevas características adicionales, fue liberado oficialmente en mayo de 2000, casi dos años después que su predecesor. Además de la mejora de rendimiento de esta versión, PHP 4.0 incluía otras características clave como el soporte para la mayoría de los servidores Web, sesiones HTTP, buffers de salida, formas más seguras de controlar las entradas de usuario y muchas nuevas construcciones de lenguaje.

El día 13 de julio de 2007 se anunció la suspensión del soporte y desarrollo de la versión 4 de PHP, aunque el soporte a fallos críticos se prolongó hasta agosto de 2008 y en enero de 2008 se liberó una última versión con mejoras de seguridad.

2.4 PHP 5

PHP 5 fue lanzado en Julio del 2004 después de un largo desarrollo y varios pre-releases. Está básicamente impulsado por su núcleo "Zend Engine 2", que contiene un nuevo modelo de objetos y docenas de nuevas opciones, como mejor soporte para MySQL, para XML (XPath, DOM, etc.), soporte nativo para SQLite, soporte integrado para SOAP, iteradores de datos, manejo de excepciones, etc. Su última versión es la 5.6.

2.5 PHP 6

Esta versión, que prometía soporte unicode y un motor optimizado que daría un mejor rendimiento, fue cancelada en 2012, aunque algunas de sus mejoras fueron integradas en PHP 5.

2.6 PHP 7

La última versión de PHP fue liberada el 3 de diciembre de 2015. Además de ofrecer soporte de 64 bits, se produjo una depuración de la sintaxis del lenguaje PHP, mejoras de seguridad y, especialmente, una importante mejora de rendimiento con el nuevo motor “Zend Engine 3”, el doble que la versión 5.6. Entre las nuevas características del lenguaje destaca el uso de las declaraciones de tipo en las funciones, lo que permite a los programadores fijar los tipos de datos de tanto los parámetros de entrada como los de salida de las funciones. Por ejemplo, en el código siguiente todos los parámetros de entrada a la función “sum” se tratarán como enteros.

```
<?php
declare(strict_types=1);
function sum(int ...$ints):string{
return array_sum($ints);
}
echo sum(1,2,"3");
echo sum(1,2,3);
?>
```

Como contrapartida a estas mejoras del lenguaje, se ha perdido compatibilidad con el lenguaje PHP5, lo que obliga en muchas ocasiones a modificar los scripts de PHP antiguos. Ejemplo de esto es que ahora obligatoriamente el código php tiene que estar incluido entre las etiquetas <?php [...] ?> y ya no se pueden utilizar las etiquetas <% [...] %> o <script language="php">[...]</script>.

La versión de PHP recomendada en la actualidad es la versión 7, pues únicamente se mantiene el soporte de la versión 5.6 hasta diciembre de 2017.

3 El lenguaje PHP

3.1 Características

El acrónimo PHP significa "PHP: Hypertext Preprocessor". Es un lenguaje de programación similar a otros lenguajes de propósito general: contiene variables, tipos de datos, sentencias de control, funciones e incluso orientación a objetos. El signo punto y coma (;) indica el final de una sentencia en PHP.

Se trata de un lenguaje de programación:

- Interpretado.
- De scripting.
- Procedimental y orientado a objetos.
- Imperativo.
- Dinámica y débilmente tipado.
- Con una sintaxis similar a C, C++, C#, Java y Perl.

3.2 Licencia PHP

La licencia PHP es la licencia bajo la que se publica el lenguaje de programación PHP. De acuerdo a la Free Software Foundation, es una licencia de software libre no copyleft, y una licencia de código abierto según la Open Source Initiative. Se permite la redistribución del contenido licenciado en forma de código fuente o binaria siempre y cuando se cumplan los siguientes requisitos:

1. Se incluya la declaración de los derechos de autor de la licencia PHP;
2. La palabra "PHP" no se use en el título de las obras derivadas;
3. Se incluya el siguiente anuncio bajo cualquier forma en la que se redistribuya el código: "This product includes PHP software, freely available from <<http://www.php.net/software/>>".

Debido a la restricción en el uso del término "PHP", no es compatible con la licencia GPL.

3.3 Páginas PHP

PHP es un lenguaje específicamente diseñado para el desarrollo de aplicaciones web en el lado del servidor.

Las páginas de PHP contienen HTML con código incrustado PHP entre las etiquetas especiales de comienzo y final `<?php` y `?>`, que permiten entrar y salir del "modo PHP". El código PHP es ejecutado en el servidor y no se mostrará al navegador cliente.

Por ejemplo, el tradicional programa "hola mundo" en PHP es:

```
<html>
<head>
  <title>Hola mundo en PHP</title>
</head>
<body>
  <?php echo '<p>Hola Mundo</p>'; ?>
</body>
</html>
```

El navegador del cliente solicitará la página PHP al servidor web, que le devolverá el código HTML siguiente:

```
<html>
<head>
  <title>Hola mundo en PHP</title>
</head>
<body>
  <p>Hola Mundo</p>
</body>
</html>
```

3.4 Funciones HTTP

PHP cuenta con funciones HTTP que permiten manipular la salida que se envía al navegador cliente a nivel de protocolo HTTP. Entre ellas se encuentra la posibilidad de enviar cabeceras HTTP y cookies al navegador cliente.

La función "header" debe llamarse antes de que se generen las etiquetas HTML. Las cookies deben enviarse antes de mandar cualquier otra cabecera (ésta es una restricción de las cookies, no de PHP). Esto requiere que se sitúen las llamadas a esta función antes de cualquier etiqueta <html>.

Como ejemplo del envío de cabeceras HTTP, este script PHP redirige el navegador cliente al sitio web de PHP:

```
<?php
header ("Location: http://www.php.net");
exit;
?>
```

La salida de este script (ejecutado con: "php-cgi.exe ejemplo.php") es:

```
Status: 302 Moved Temporarily
X-Powered-By: PHP/5.6.4
Location: http://www.php.net
Content-type: text/html; charset=UTF-8
```

Los scripts de PHP a menudo generan HTML dinámico que no debe almacenarse en la caché del navegador cliente o en la caché de cualquier proxy situado entre el servidor y el navegador cliente. Se puede obligar a que deshabiliten el almacenamiento en caché mediante cabeceras HTTP como "Pragma: no-cache" (HTTP/1.0) o "Cache-Control: no-cache, must-revalidate" (HTTP/1.1.).

Como ejemplo del envío de cookies, este script PHP hace uso de la función "setcookie" para definir la cookie "Testcookie" que será enviada al navegador cliente con el resto de la información de la cabecera. La cookie expirará en una hora.

```
<?php
setcookie("Testcookie","Testvalue", time()+3600);
echo $Testcookie;
?>
```

La salida de este script (ejecutado con: "php-cgi.exe ejemplo.php") es:

X-Powered-By: PHP/5.6.4

Set-Cookie: Testcookie=Testvalue; expires=Sun, 10-Sep-2017 08:14:17 GMT; Max-Age=3600

Content-type: text/html; charset=UTF-8

3.5 Constantes

Una constante es un identificador para un valor que no se puede cambiar durante la ejecución.

Un nombre de constante válido empieza por una letra o guión bajo, seguido por cualquier número de letras, números o guiones bajos. Las constantes se declaran mediante la palabra clave “const” o mediante la función “define()”. La diferencia entre ambas es que “const” define las constantes en tiempo de compilación, mientras que “define()” lo hace en tiempo de ejecución. A continuación se muestra un ejemplo de declaración de constantes:

```
<?php
const minvalor=1.0;
define ("maxvalor",2);
echo minvalor;
print maxvalor;
?>
```

Como se verá más adelante, las constantes, al contrario que las variables, no son precedidas por el signo dólar.

3.6 Variables

Las variables en PHP van precedidas por un signo dólar (\$), como en Perl. El nombre de la variable es sensible a minúsculas y mayúsculas. Los nombres de variables siguen las mismas reglas que otras etiquetas en PHP. Un nombre de variable válido tiene que empezar con una letra o un carácter de subrayado (_), seguido de cualquier número de letras, números y caracteres de subrayado. Por ejemplo, \$x o \$_12 son nombres de variables válidos, mientras que \$12 o \$-12 no lo son.

La asignación de valores a una variable se realiza con el operador de asignación igual (=). Por ejemplo, para asignar el valor 800 a la variable x se escribe: \$x=800;

El ámbito de una variable es el contexto dentro del que la variable está definida. La mayor parte de las variables PHP sólo tienen un ámbito simple. Este ámbito simple también abarca los ficheros incluidos y los requeridos. Por ejemplo:

```
<?php
$a = 1;
include 'b.inc';
?>
```

Aquí, la variable \$a estará disponible en el interior del script incluido “b.inc”. Sin embargo, al interior de las funciones definidas por el usuario se introduce un ámbito local a la función. Cualquier variable usada dentro de una función está, por omisión, limitada al ámbito local de la función. Dentro de una función se pueden acceder a las variables globales mediante la palabra clave “global”, como se muestra en el siguiente ejemplo:

```
<?php
$a = 1;
$b = 2;

function Suma()
{
    global $a, $b;

    $b = $a + $b;
}

Suma();
echo $b;
?>
```

El script anterior producirá la salida 3. Al declarar \$a y \$b globales dentro de la función, todas las referencias a tales variables se referirán a la versión global. No hay límite al número de variables globales que se pueden manipular dentro de una función.

PHP permite utilizar variables dinámicas, que son variables cuyo nombre es el contenido de otra variable. Por ejemplo, \$nombre="pepe"; es equivalente a: \$temp="nombre";\$\$temp="pepe"; donde PHP crea una variable \$nombre cuyo valor es "pepe". Si queremos imprimir por pantalla el contenido de la variable \$nombre escribiríamos: print "\${\$temp}"; que mostraría por pantalla "pepe". La utilidad de las variables dinámicas es que permiten crear un número elevado de variables de forma automática en un bucle, o utilizando un operador de concatenación.

Aunque normalmente las variables se asignan por valor, en PHP a partir de la versión 4 se puede modificar este comportamiento, de tal manera que la asignación no se limite a copiar el valor de una variable a otra, sino que las enlace para que un cambio en cualquiera de ellas afecte a las dos. Para ello se utiliza el carácter ampersand (&), que indica "la dirección de memoria apuntada por". Es decir, la asignación por referencia consigue que una variable apunte a la misma zona de memoria apuntada por otra. Por ejemplo, si escribimos: \$origen=666; \$destino=&\$origen; ahora la variable \$destino tiene el valor 666, y si cambiamos el valor de \$origen, también cambia el valor de \$destino ya que la asignación se ha realizado por referencia.

PHP proporciona una gran cantidad de variables predefinidas para todos los scripts. Las variables representan de todo, desde variables externas hasta variables de entorno incorporadas, desde los últimos mensajes de error hasta los últimos encabezados recuperados.

Algunas variables predefinidas en PHP son "superglobales", lo que significa que están disponibles en todos los ámbitos a lo largo del script. No es necesario emplear "global \$variable;" para acceder a ellas dentro de las funciones o métodos. Las variables superglobales son:

- \$GLOBALS: Hace referencia a todas las variables disponibles en el ámbito global.
- \$_SERVER: Información del entorno del servidor y de ejecución.
- \$_GET: Variables HTTP GET.
- \$_POST: Variables HTTP POST.
- \$_FILES: Variables de subida de ficheros HTTP.
- \$_COOKIE: Cookies HTTP.
- \$_SESSION: Variables de sesión.
- \$_REQUEST: Variables HTTP Request
- \$_ENV: Variables de entorno.

Como ejemplo del uso de las variables superglobales, el siguiente código muestra el software del servidor sobre el que se ejecuta la página PHP:

```
<?php
echo $_SERVER['SERVER_SOFTWARE'];
?>
```

3.7 Tipos de datos

Los tipos de datos permiten indicar qué clase de valor va a almacenar una variable, lo que es importante para utilizar adecuadamente la memoria disponible. En PHP las variables pueden usarse de manera muy flexible, ya que se establece el tipo de una variable al realizar una asignación. Por lo tanto, una variable puede contener al principio del programa un entero, y posteriormente ser utilizada para almacenar una cadena de texto. Entre los tipos de datos básicos de PHP se encuentran: integer (números enteros), double (número de coma flotante), string (cadena de texto) y boolean (sólo dos valores permitidos: verdadero o falso).

Además de estos tipos básicos, PHP dispone de arrays y de objetos. Un array es un conjunto de variables de un mismo tipo, indexadas por un número o una cadena de texto (array asociativo). Un ejemplo de array es: `$alumno[1]="Epi"; $alumno[2]="Blas";` y un ejemplo de array asociativo: `$alumno[nombre]="Blas";`

3.8 Operadores

En PHP, al igual que en otros lenguajes de programación, existen diferentes tipos de operadores. Los operadores toman uno o más valores y producen otro valor.

El operador de asignación básico es “=”.

Los operadores aritméticos son adición (+), sustracción (-), multiplicación (*), división (/), módulo (%), exponenciación (**), identidad (+) y negación (-).

Los operadores de comparación son igual (==), idéntico (===), diferente (!=, <>), no idéntico (!==), menor que (<), mayor que (>), menor o igual que (<=), mayor o igual que (>=).

Los operadores lógicos son And (and, &&), Or (or, ||), Xor (xor), Not (!).

En el siguiente ejemplo, se imprime por pantalla el valor “x es 1”:

```
$x=2;$y=3;++$x;$x>=$y&&$x<4?$x=$x%2:$x=$y*2;print "x es $x";
```

3.9 Sentencias de control

PHP también dispone de sentencias de control, bien sean estructuras condicionales o estructuras iterativas.

Entre las primeras se encuentra “if”, de tal manera que en “if(expresión) {sentencias;}” sólo se ejecutarán las sentencias si “expresión” es verdadera.

Dentro de las estructuras iterativas se encuentra “while”, de tal manera que “while(expresion){sentencias;}” ejecutará las sentencias mientras “expresión” sea verdadera. La expresión se evalúa cada vez que se inicia una iteración del bucle. Ésa es la diferencia con la sentencia “do..while”, donde la condición es comprobada al final de cada iteración en lugar de hacerlo

al principio. En la sentencia “for”, del tipo “for(inicialización;condición;modificación){sentencias}”, las sentencias se ejecutarán mientras “condición” sea cierta, y la evaluación se realiza al comienzo de cada iteración, al igual que en la sentencia “while”.

A continuación se muestra un sencillo bucle que calcula la suma de los primeros 10 números naturales. En este ejemplo, se utiliza una variable \$suma en la que se irán acumulando las sumas parciales y una variable \$x que actuará de contador en el bucle. La variable contador se inicializa con el valor 1, y mientras su valor sea menor o igual que 10 se seguirá ejecutando el bucle. Al finalizar cada iteración, la variable contador se incrementa en una unidad. El cuerpo del bucle suma el valor de la variable contador a la suma parcial acumulada hasta el momento.

```
$suma=0;
for ($x=1;$x<=10;$x++){
    $suma=$suma+$x;
}
```

PHP4 añadió la sentencia de construcción de bucles “foreach”, que permite recorrer todos los elementos de un array utilizando esta sintaxis: “foreach (nombre_array as elemento_array){sentencias}”. A continuación se muestra un ejemplo de esta sentencia.

```
$mi_array=array("coche","casa");
foreach ($mi_array as $cosa){
    print "El elemento es $cosa. <BR>";
}
```

Su ejecución genera los siguientes mensajes en la página:

El elemento es coche.
El elemento es casa.

3.10 Orientación a objetos

Mediante la programación orientada a objetos se pueden definir en el programa Clases, que contendrán variables y funciones (métodos). Una vez definida una clase y a partir de ella, será posible instanciar un objeto. La definición de una clase en PHP comienza con la palabra clave “class” y las variables de la clase se declaran con la palabra clave “var”. Los métodos de la clase se declaran mediante la palabra clave “function”. La sintaxis para la creación de un nuevo objeto de una clase es: “\$objeto=new nombredelaclase(argumentos)”.

En este ejemplo, se define en el fichero “verdura.inc” la clase base “verdura” que describe una verdura en general, si es comestible o no y cuál es su color.:

```
<?php
class Verdura {

    var $comestible;
    var $color;

    function Verdura($comestible, $color="verde")
    {
        $this->comestible = $comestible;
        $this->color = $color;
    }
}
```

```
}

function es_comestible()
{
    return $this->comestible;
}

function qué_color()
{
    return $this->color;
}
}
?>
```

A continuación se utilizará la clase anterior en una página PHP que imprimirá por pantalla: "vegetariano: CLASE Verdura":

```
<pre>
<?php
include "verdura.inc";
$vegetariano = new Verdura(true, "blue");
echo "vegetariano: CLASE " . get_class($vegetariano) . "\n";
?>
</pre>
```

PHP se reserva todos los nombres de los métodos que comienzan con __ como mágicos. Los nombres de los métodos __construct(), __destruct(), __call(), __callStatic(), __get(), __set(), __isset(), __unset(), __sleep(), __wakeup(), __toString(), __invoke(), __set_state(), __clone() y __debugInfo() son mágicos en las clases PHP. No se puede tener métodos con estos nombres en ninguna clase a menos que se desee la funcionalidad mágica asociada a estos. Por ejemplo, el método __toString() permite a una clase decidir cómo comportarse cuando se le trata como un string. En el código siguiente se puede ver un ejemplo de este método mágico:

```
<?php
class TestClass
{
    public $foo;

    public function __construct($foo)
    {
        $this->foo = $foo;
    }

    public function __toString()
    {
        return $this->foo;
    }
}

$class = new TestClass('Hola Mundo');
echo $class;
?>
```

El resultado de ejecutar este código es "Hola Mundo".

3.11 Funciones

Las funciones en PHP se declaran mediante la palabra clave “function”. Cabe destacar que PHP, al igual que otros lenguajes como Javascript, permite el uso de funciones anónimas. Las *funciones anónimas* son funciones que no tienen un nombre especificado. Una *lambda* es una función anónima que puede ser asignada a una variable o pasada como argumento a otra función.

El siguiente es un ejemplo de función lambda, donde se asigna una función anónima a la variable \$saludo:

```
<?php
$saludo = function() {
    return "Hola que tal";
};
echo $saludo(); // Devuelve: Hola que tal
?>
```

Este es un ejemplo de una función lambda que se pasa como argumento a la función “shout”:

```
<?php
function shout ($message){
    echo $message(); // Devuelve: Hola que tal
}
// Call function
shout(function(){
    return "Hola que tal";
});
?>
```

Las funciones lambda también pueden ser “funciones de usar y tirar”, como se ve en este ejemplo:

```
<?php
// Use create_function
$greeting = create_function('', 'echo "Hello World!";');

// Call function
$greeting();
?>
```

Una *clausura* o closure es una función anónima que captura el contexto o *scope* actual, y proporciona acceso a ese contexto cuando se invoca el closure.

```
<?php
$colorCoche = 'rojo';
$mostrarColor = function() use ($colorCoche) {
    echo "El color del coche es $colorCoche";
};
$mostrarColor(); // Mostrará El color del coche es rojo
?>
```

3.12 Tratamiento de excepciones

PHP tiene un modelo de excepciones similar al de otros lenguajes de programación. Una excepción puede ser lanzada ("thrown"), y atrapada ("caught") dentro de PHP. El código puede estar dentro de un bloque try para facilitar la captura de excepciones potenciales. Cada bloque try debe tener al menos un bloque catch o finally correspondiente.

A continuación se muestra un ejemplo. La función "inverso" lanza una excepción que es recogida mediante la sentencia "catch":

```
function inverso($x){
    if (!$x){
        throw new Exception('División por cero');
    }
    return 1/$x;
}
try{
    echo inverso(5)."\n";
    echo inverso(0)."\n";
}
catch(Exception $e)
{
    echo 'Excepción capturada: '.$e->getMessage()."\n";
}
```

3.13 Conexión con bases de datos

Si una aplicación de PHP necesita comunicarse con un servidor de bases de datos, será necesario escribir código de PHP para llevar a cabo actividades como conectarse al servidor de bases de datos, consultar la base de datos y realizar otras funciones relacionadas con la base de datos. Un controlador (o *driver*) es una pieza de software diseñada para la comunicación con un tipo específico de servidor de bases de datos. El controlador también llama a una biblioteca, como la Biblioteca Cliente de MySQL o el Controlador Nativo de MySQL. Estas bibliotecas implementan el protocolo de bajo nivel usado para comunicarse con el servidor de bases de datos.

Para conectarse desde PHP a los servidores de bases de datos, se pueden utilizar dos interfaces:

- **PDO** (Objetos de Datos de PHP): es la capa de abstracción de bases de datos que puede usar uno de los varios controladores específicos de bases de datos. Uno de los controladores disponibles es el controlador de MySQL de PDO, que permite hacer de interfaz con el servidor MySQL. PDO ofrece una API homogénea para las aplicaciones PHP, independientemente del tipo de servidor de bases de datos con el que se vaya a conectar la aplicación. Si se utiliza la API PDO, se podría cambiar el servidor de bases de datos en uso, por ejemplo de Firebird a MySQL, y sólo se necesitarían algunos cambios menores en el código PHP. PDO está desarrollado utilizando el framework de extensiones de PHP. Su mayor inconveniente es que no se pueden utilizar todas las funcionalidades avanzadas del sistema gestor de bases de datos. En la siguiente figura se observa que PDO es una capa de acceso a múltiples bases de datos:



Figura 1. PDO

- *Extensiones de bases de datos específicas del proveedor:* El código de PHP consiste en un núcleo, con extensiones opcionales para la funcionalidad del núcleo. Las extensiones relacionadas con MySQL de PHP, como la extensión mysqli, y la extensión mysql, están implementadas con el marco de extensiones de PHP. Una extensión típicamente expone una API al programador de PHP para así utilizar sus prestaciones programáticamente. Sin embargo, algunas extensiones que usan el marco de extensiones de PHP no exponen una API al programador de PHP. La extensión del controlador de MySQL de PDO, por ejemplo, no expone una API al programador de PHP, pero proporciona una interfaz para la capa superior a PDO. Las extensiones de bases de datos que se pueden utilizar son:

- CUBRID
- DB++
- dBase
- filePro
- Firebird/InterBase
- FrontBase
- IBM DB2 — IBM DB2, Cloudscape y Apache Derby
- Informix
- Ingres — Ingres DBMS, EDBC, y Enterprise Access Gateways
- MaxDB
- Mongo — Controlador de MongoDB (antiguo)
- MongoDB — Controlador de MongoDB
- mSQL
- Mssql — Microsoft SQL Server
- MySQL — Controladores y complementos de MySQL
- OCI8 — Oracle OCI8
- Paradox — Acceso a archivos Paradox
- PostgreSQL
- SQLite
- SQLite3
- SQLSRV — Driver de Microsoft SQL Server para PHP
- Sybase
- tokyo_tyrant

Por ejemplo, para MySQL, PHP ofrece tres API diferentes para conectarse a MySQL:

- mysql (API original de MySQL)
- mysqli (Extensión MySQL mejorada)
- PDO (Objetos de Datos de PHP)

En el siguiente ejemplo, cada trozo de código crea una conexión al servidor de MySQL que se está ejecutando en "ejemplo.com" con el nombre de usuario "usuario" y la contraseña "contraseña". También se ejecuta una consulta para saludar al usuario.

```
<?php
// mysqli
$mysqli = new mysqli("ejemplo.com", "usuario", "contraseña", "basedatos");
$resultado = $mysqli->query("SELECT '¡Hola, usuario de MySQL!' AS _message FROM DUAL");
$fila = $resultado->fetch_assoc();
echo htmlentities($fila['_message']);

// PDO
$pdo = new PDO('mysql:host=ejemplo.com;dbname=basedatos', 'usuario', 'contraseña');
$sentencia = $pdo->query("SELECT '¡Hola, usuario de MySQL!' AS _message FROM DUAL");
$fila = $sentencia->fetch(PDO::FETCH_ASSOC);
echo htmlentities($fila['_message']);

// mysql
$c = mysql_connect("ejemplo.com", "usuario", "contraseña");
mysql_select_db("basedatos");
$resultado = mysql_query("SELECT '¡Hola, usuario de MySQL!' AS _message FROM DUAL");
$fila = mysql_fetch_assoc($resultado);
echo htmlentities($fila['_message']);
?>
```

Se recomienda usar las extensiones `mysqli` o `PDO_MySQL`. No se recomienda usar la extensión `mysql` antigua para nuevos desarrollos, ya que ha sido declarada obsoleta en PHP 5.5.0 y eliminada en PHP 7. También señalar que para administrar las instancias de MySQL se puede utilizar la aplicación web `phpMyAdmin`.

3.14 PHP-CLI

Aunque el lenguaje PHP está ligado al desarrollo de aplicaciones web, también se pueden realizar con él aplicaciones “standalone” basadas en scripts. Fue a partir de la versión 4.3 cuando se introdujo el intérprete de línea de comandos PHP-CLI. Un sencillo programa “hola mundo” sería algo así:

```
#!/usr/bin/php
<?php
echo "Hola mundo \n";
exit(0);
?>
```

La primera línea permite al usuario etiquetar el fichero como ejecutable y poder lanzarlo como un script de consola (“./holamundo.php”). También se puede ejecutar escribiendo: “php holamundo.php”. La salida por consola es: “Hola mundo”;

Hay que tener en cuenta que los intérpretes de PHP para línea de comandos y para interfaces CGI, que utilizan los servidores web para lanzar programas externos, son diferentes. En el primer caso, el binario es “php” y en el segundo caso es “php-cgi”. El intérprete CGI siempre devuelve una cabecera HTTP, con lo cual la salida del script “hola mundo” al ejecutarlo mediante “php-cgi holamundo.php” es:

```
X-Powered-By: PHP/5.6.30-0+deb8u1
Content-type: text/html; charset=UTF-8
```

Hola mundo

4 Entornos PHP

4.1 Tecnologías para la ejecución de PHP

Como se ha señalado anteriormente, el lenguaje PHP permite realizar aplicaciones web o aplicaciones standalone basadas en scripts.

Respecto a las aplicaciones web, dependiendo de cómo se ejecute el código PHP, se pueden distinguir tres modalidades: ejecución dentro del servidor web mediante un módulo PHP o ejecución externa mediante CGI o FastCGI.

En la primera modalidad, las páginas PHP son ejecutadas por un módulo del servidor web que embebe el intérprete PHP dentro de cada proceso del servidor web. Es decir, el intérprete de PHP se ejecuta junto con el servidor web. Por ejemplo, en la lista de módulos cargados por Apache ("apachectl -M") aparecería "php5_module (shared)". El uso de un módulo PHP en el servidor web es muy recomendable cuando la cantidad de código a ejecutar por el servidor web es muy alta. La desventaja es que consume más recursos del sistema. Ésta es la modalidad utilizada en aplicaciones web con muchas páginas PHP, como es, por ejemplo, WordPress.

Por su parte, en las aplicaciones basadas en scripts CGI la ejecución del código PHP se realiza de manera externa al servidor web. Tiene ventajas desde el punto de vista de la seguridad, ya que el contexto de seguridad de la ejecución del código PHP es distinto que el del servidor web, y desde el punto de vista del rendimiento, pues el intérprete sólo se utiliza cuando hace falta. Las aplicaciones CGI corren en un proceso separado que es creado al inicio de cada petición y es finalizado cuando termina su trabajo. Este modelo de "un nuevo proceso por cada petición" hace que los programas CGI sean muy fáciles de implementar, pero limita la eficiencia y la escalabilidad. En altas cargas, la sobrecarga en el sistema operativo por la creación y destrucción de procesos se torna significativa y limita la escalabilidad. Además, el modelo de procesos CGI limita las técnicas de reutilización de recursos (como la reutilización de conexiones a bases de datos, caché en memoria, etc), que si están presentes cuando el intérprete de PHP está embebido en el servidor web.

Hay un enfoque intermedio que es el adoptado por FastCGI, que intenta solventar la principal desventaja de CGI, y reducir la carga asociada con el hecho de interconectar el servidor web y los programas CGI permitiéndole a un servidor atender más peticiones a la vez. En vez de crear procesos nuevos por cada petición, FastCGI puede usar un solo proceso persistente que maneja cualquier petición durante su período de vida. El hecho de procesar múltiples peticiones a la vez es logrado ya sea mediante la utilización de una sola conexión con un multiplexado interno (por ejemplo, múltiples peticiones sobre una sola conexión) y/o utilizando múltiples conexiones. Varios de esos procesos pueden existir, y eso es algo que incrementa la escalabilidad y el rendimiento. FastCGI permite también a los programas hacer que el servidor web realice ciertas operaciones sencillas, como leer un archivo antes de que la petición sea procesada. Las variables de entorno y las peticiones de páginas son enviadas desde el servidor web hacia los procesos a través de una conexión TCP (para procesos remotos) o a través de sockets de Unix (para procesos locales). Las respuestas son devueltas desde el proceso hacia el servidor web sobre la misma conexión. La conexión puede ser cerrada al final de una respuesta, pero el servidor web y el proceso se mantienen en ejecución. Por ejemplo, en el caso de hospedar una aplicación PHP en IIS, se puede utilizar como alternativa al módulo ISAPI de PHP el módulo FastCGI, que creará un pool de conexiones al ejecutable "php-cgi.exe", que atenderá las peticiones PHP.

4.2 Arquitectura LAMP

En la siguiente figura se muestra la arquitectura de capas (“tiers”) de las aplicaciones web PHP.

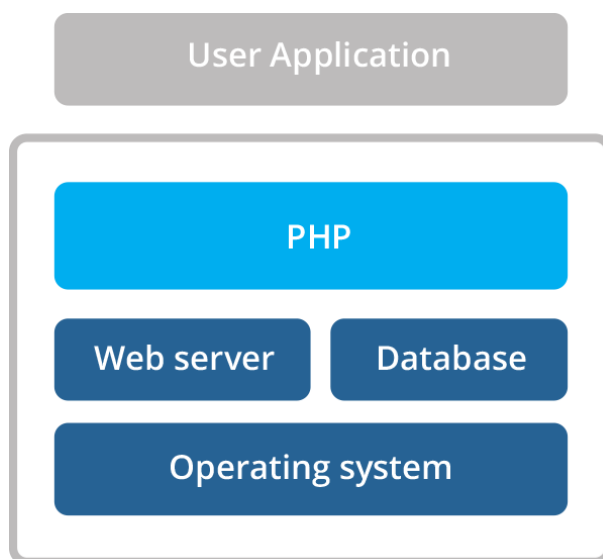


Figura 2. Arquitectura de aplicaciones web PHP

Las aplicaciones PHP se despliegan en un servidor web, que puede ser Apache, lighttpd, Nginx, Zend Server, IIS, etc. Este servidor tendrá que tener un módulo PHP. También tiene que estar habilitado el soporte para el sistema gestor de Bases de Datos que se vaya a utilizar. El sistema operativo sobre el que se ejecuta puede ser Linux, Windows, o macOS.

Una arquitectura de aplicaciones PHP típica es la descrita por los acrónimos LAMP (Linux+Apache+MySQL/MariaDB+PHP) o WAMP (Windows+Apache+MySQL/ MariaDB+PHP).

Un servidor LAMP es un caso particular de servidor de aplicaciones. Los servidores LAMP presentan dos características principales:

- El gran dinamismo de los contenidos y servicios que se ofrecen.
- Su fuerte integración con bases de datos.

Los servidores LAMP son muy populares gracias a que las diferentes herramientas que integran la solución global son software libre, proporcionando una plataforma de desarrollo de bajo coste con una gran funcionalidad y alta capacidad de parametrización. Su uso es muy común para desarrollar portales web o plataformas de comercio electrónico, ya que permiten desde cualquier dispositivo el acceso a los servicios web de una forma segura, transparente y centralizada.

Por ejemplo, para montar un servidor LAMP sobre Linux Debian habría que ejecutar los siguientes comandos:

```
sudo apt-get install apache2 php5 libapache2-mod-php5  
sudo apt-get install mysql-server mysql-client php5-mysql
```

Como ejemplo de las pilas de aplicaciones PHP profesionales, se puede señalar, según la figura 2, Debian, Apache, MySQL/MariaDB/SQLite, PHP5, ownCloud/WordPress.

4.3 Frameworks de desarrollo

La aplicación PHP es un conjunto de páginas PHP, formularios HTML y demás recursos como ficheros, librerías Javascript o imágenes. Como se ha señalado anteriormente, una página PHP consiste en código PHP incrustado dentro de código HTML.

Dentro de la aplicación, habrá páginas que implementen la lógica de presentación, otras la lógica de aplicación y otras la lógica de acceso a datos. Por ejemplo, la capa de presentación puede consistir en un formulario HTML que contenga un método GET/POST cuya acción sea procesada por una página PHP. Esta página PHP recibirá los datos introducidos por el usuario para su procesamiento. Para ello puede hacer uso de las variables superglobales `$_REQUEST` o `$_GET` o `$_POST`, según el método HTTP que se haya utilizado en el formulario. De hecho, una de las características más destacadas del lenguaje PHP es la sencillez y la facilidad de interacción con los formularios HTML.

Este ejemplo explica la facilidad cómo PHP procesa una petición HTTP y devuelve una respuesta HTTP:

```
$uri = $_SERVER['REQUEST_URI'];
$foo = $_GET['foo'];
header('Content-Type: text/html');
echo 'The URI requested is: '.$uri;
echo 'The value of the "foo" parameter is: '.$foo;
```

Lo que el navegador web del cliente recibe es:

```
HTTP/1.1 200 OK
Date: Sat, 03 Apr 2011 02:14:33 GMT
Server: Apache/2.2.17 (Unix)
Content-Type: text/html
The URI requested is: /testing?foo=a
The value of the "foo" parameter is: a
```

Tradicionalmente, las aplicaciones web PHP se construían como un conjunto de páginas PHP que recibían cada una directamente las peticiones, y así, la página `tarea1.php` ejecutaba la tarea 1 y era llamada mediante `/tarea1.php`; la página `tarea2.php` ejecutaba la tarea 2 y era llamada mediante `/tarea2.php`, y así sucesivamente. Este enfoque se demostró ineficiente pues, entre otros inconvenientes, cada página PHP tiene que cargar todo el código y los recursos necesarios para las funcionalidades comunes a todas las tareas como son el acceso a bases de datos, seguridad, presentación, estilos, etc.

Al igual que en otros entornos de desarrollo de aplicaciones como Java o .NET, en PHP también existen frameworks para el desarrollo de aplicaciones web, que facilitan la adopción de patrones como MVC (Modelo–vista–controlador) y permiten el desarrollo ágil de aplicaciones. Entre ellos, se pueden destacar Zend Framework, Symfony y Laravel.

Zend Framework (ZF) está liberado bajo la licencia BSD y Symfony bajo la licencia MIT. Aunque ambos implementan el paradigma MVC, Symfony está más orientado que ZF para la creación de sitios web con muchos usuarios, y es utilizado por CMS como Drupal. Respecto al soporte ORM (mapeo de objetos a bases de datos relacionales), necesario porque los frameworks están orientados a objetos, Symfony integra plugins ORM como Propel o Doctrine, mientras que ZF necesita librerías de terceras partes.

Laravel es el más moderno de los tres, fue creado en 2011. Utiliza componentes de Symfony. Tiene como objetivo ser un framework que permita el uso de una sintaxis elegante y expresiva para crear código de forma sencilla, al tiempo que ofrece muchas funcionalidades. Utiliza un sistema de

plantillas denominado “Blade”, que permite una sintaxis más reducida. Las plantillas en Blade son archivos de texto plano que contienen el código HTML de la página con etiquetas que representan elementos, zonas o vistas parciales a incluir en la plantilla. Por ejemplo, una plantilla de Blade puede incluir código como:

```
<html>
<head>
<title>@yield('titulo')</title>
</head>
</html>
```

Donde “@yield()” identifica el método para desplegar la zona llamada “titulo”, que estará definida en una vista –por ejemplo, “vista.blade.php”- mediante “@section(‘titulo’)”.

Al igual que los otros frameworks, Laravel dispone de una clase base controlador para ser utilizada para crear un controlador que, como se comentará en el siguiente párrafo, contiene la lógica de la aplicación.

Un patrón de diseño muy común en las aplicaciones web PHP es el patrón “Front controller”. Se utiliza una página PHP como controlador frontal, de tal manera que todas las peticiones pasan por él. Siguiendo el ejemplo anterior, las peticiones “/controlador.php/tarea1” o “/controlador.php/tarea2” son atendidas por la página “controlador.php”, quien ya se encargará de realizar las llamadas necesarias a “tarea1.php” y “tarea2.php” de manera transparente. La adopción de un controlador frontal se observa en muchas aplicaciones profesionales; por ejemplo, en Owncloud el URI para el acceso a los ficheros es “index.php/apps/files/”, o en WordPress: “index.php/2017/08/03/hola-mundo/”. También se pueden omitir las extensiones .php en las URLs mediante las reglas de reescritura definidas en el servidor web.

En la siguiente figura se muestra la arquitectura del framework Symfony.

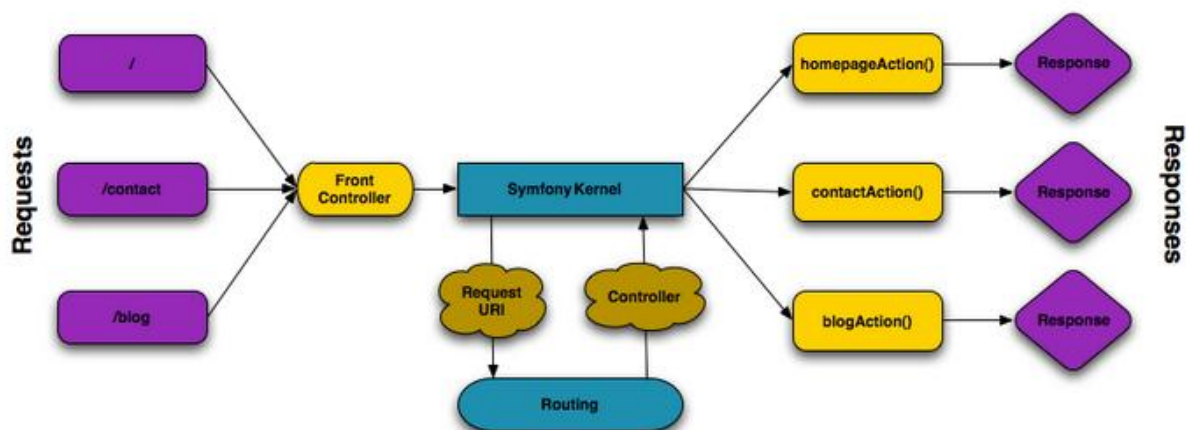


Figura 3. Arquitectura de Symfony

Existe un fichero de configuración del enrutamiento que mapea las diferentes URLs con las funciones PHP. La tarea de cada función PHP -denominada por Symfony controlador-, es utilizar la información procedente de las peticiones para crear una respuesta. El controlador frontal simplemente recibe las peticiones y Symfony, con la configuración de enrutamiento, sabe que controlador (función) se tiene que ejecutar, devolviendo al cliente las cabeceras HTTP y la respuesta. Este framework también proporciona funcionalidades como cacheo HTTP y el acceso a herramientas de software libre como Varnish (un acelerador HTTP).

4.4 Entornos de desarrollo integrado

Existen diferentes entornos de desarrollo integrado (IDE) en PHP. Dos de los más populares son Zend Studio y Eclipse (PHP Development Tools). La principal ventaja de estos entornos es que facilitan el desarrollo de las aplicaciones, pues disponen de funcionalidades útiles para el programador como la capacidad de resaltar y validar la sintaxis del lenguaje, o la disponibilidad de plantillas de código. Como componentes básicos, tienen un editor de código, un compilador/intérprete y un depurador, aunque las funcionalidades se pueden ampliar con “plugins”. Asimismo, se integran con los servidores de aplicaciones para desplegar y probar las aplicaciones.

4.5 Aceleradores PHP

Los aceleradores PHP son componentes diseñados para aumentar el rendimiento de las aplicaciones PHP. Sin un acelerador PHP, en cada petición que se haga al servidor web, el código PHP se compilará a bytecodes y luego será interpretado. Es decir, el código PHP necesita ser recompilado y reejecutado en cada petición. Con un acelerador PHP sólo se necesita reejecutar el código. Los aceleradores no son la única manera de mejorar el rendimiento de PHP. Por ejemplo, Facebook ha desarrollado el transcompilador HipHop for PHP (HHpHPc), que permite traducir el código PHP a C++, que posteriormente es compilado.

La mayoría de los aceleradores de PHP trabajan almacenando en caché scripts PHP compilados en bytecode (caché “opcode”) para evitar la sobrecarga de compilación del código en cada solicitud. Un acelerador PHP generalmente reduce la carga del servidor y aumenta la velocidad de código PHP entre 2 y 10 veces, dependiendo de factores tales como el tiempo de ejecución inherentes a la aplicación PHP y el porcentaje de código fuente que realmente es ejecutado en una solicitud determinada. Normalmente los aceleradores PHP aparte de cachear el código lo optimizan, con lo cual el rendimiento en la ejecución es todavía mejor.

Como ejemplos de aceleradores PHP se pueden señalar: Alternative PHP Cache, IonCube PHP Accelerator, XCache, Turck MMCache, Zend OPcache, Zend Platform, NuspHERE PhpExpress, o Windows Cache Extension for PHP.

ZendOPcache es la caché “opcode” incluida con las versiones de PHP superiores a 5.5, como se observa al ejecutar el comando “php -v”:

```
PHP 5.6.30-0+deb8u1 (cli) (built: Apr 14 2017 16:20:58)
Copyright (c) 1997-2016 The PHP Group
Zend Engine v2.6.0, Copyright (c) 1998-2016 Zend Technologies
    with Zend OPcache v7.0.6-dev, Copyright (c) 1999-2016, by Zend Technologies
```

Otros aceleradores como Alternative PHP Cache (APC) no sólo funcionan como caché “opcode” sino también como caché de datos o de objetos, con lo cual están recomendados para funcionar con aplicaciones como WordPress o Drupal.

Frente a los aceleradores PHP implementados como extensiones de PHP, también se pueden utilizar aceleradores HTTP configurados para cachear el código PHP. Éste es el caso del citado Varnish. Funciona como un proxy inverso -delante del servidor web, por ejemplo Apache- que almacena ficheros o fragmentos de los mismos en memoria para reducir el tiempo de respuesta y el consumo de ancho banda, y además puede usarse como Firewall de Aplicaciones, Balanceador de Carga o para prevenir ataques DDoS.

GLOSARIO

CGI: Common Gateway Interface.
CMS: Content Management System.
DDoS: Denial-of-service attack.
HTML: HyperText Markup Language.
HTTP: Hypertext Transfer Protocol.
IDE: Integrated Development Environment.
IIS: Internet Information Services.
ISAPI: Internet Services API.
MVC: Model-View-Controller.
ORM: Object Relational Mapper.
PDO: PHP Data Objects.
URI: Uniform Resource Identifier.

RESUMEN ESQUEMÁTICO

- Evolución de PHP: versiones 3,4,5,6.7. Soporte sólo para 5 y 7.
- Lenguaje:
 - características (interpretado, embebido en HTML, scripting, sentencia;)
 - constantes (const, define())
 - variables (\$x, dinámicas, globales, superglobales)
 - bucles (for, foreach, while)
 - operadores (aritméticos, comparación, lógicos)
 - sentencias condicionales (if)
 - tratamiento de excepciones (try, catch)
 - orientación a objetos (clases)
 - funciones (anónimas, lambda)
 - funciones HTTP (cabeceras, cookies)
 - acceso a bases de datos (PDO, específicas (mysqli))
- Entornos:
 - arquitectura LAMP.
 - ejecución como módulo del servidor web o CGI/FastCGI,
 - frameworks (Laravel, Symfony, Zend): patrones de diseño.
 - IDEs (Eclipse, Zend).
 - Aceleradores (por defecto ZendOpCache).

PREGUNTAS DE TEST

- 1) ¿Cuál de los siguientes bloques de código en PHP permite que se escriba al menos una vez el valor de la variable \$i?
 - a) `<?php $i=0; do {echo $i;} while ($i>0);?>`
 - b) `<?php $i=1;while($i<0){echo $i;}?>`
 - c) `<?php for ($i=1;$i<0;$i--){echo $i;}?>`
 - d) `<?php $i=1;if($i<0){echo $i;}?>`

- 2) ¿Cuáles de los siguientes mecanismos no existe específicamente como caché de código PHP?
 - a) Memcached
 - b) OpCache
 - c) WinCache Extension for PHP
 - d) Alternative PHP Cache

- 3) Sobre la declaración de variables en el lenguaje de programación PHP:
 - a) Los nombres de las variables pueden comenzar por cualquier carácter, incluidos números y guión bajo.
 - b) Las variables comienzan con el símbolo \$.
 - c) Es necesario declarar las variables y establecer el tipo de datos que lo almacenará antes de usarlas.
 - d) Para declarar una variable de ámbito global, hay que anteponer la palabra `#define`.

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	a
2	a
3	b

BIBLIOGRAFÍA BÁSICA

1. <http://php.net>

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

2. <https://github.com/Maikuolan/php-knowledge>
3. Symfony: The Book. 2016.
4. "Learning PHP", O'Reilly Media, 2016.